Solana

# Send a transaction

Open in Claude

Environment

React

To rely on Privy's API to fill in the recentBlockhash field of the Solana transaction, pass in the dummy value `11111111111111111111111111111111` for the recentBlockhash field.

When using Privy's server-side SDKs to send transactions, you can use the authorization context to automatically sign requests. Learn more about **signing on the server**.

To send a transaction from a wallet using the React SDK, use the `signAndSendTransaction` method from the `useSignAndSendTransaction` hook:

```
signAndSendTransaction: (input: {
  transaction: Uint8Array;
  wallet: ConnectedStandardSolanaWallet;
  chain?: SolanaChain;
  options?: SolanaSignAndSendTransactionOptions & {
    uiOptions?: SendTransactionModalUIOptions;
    sponsor?: boolean;
  };
}) => Promise<{ signature: Uint8Array }>
```

## Usage

```
import {useSignAndSendTransaction, useWallets} from '@privy-io/reac
import {
  pipe,
  createSolanaRpc,
  getTransactionEncoder,
  createTransactionMessage,
  setTransactionMessageFeePayer,
  setTransactionMessageLifetimeUsingBlockhash,
  appendTransactionMessageInstructions,
  compileTransaction,
  address,
  createNoopSigner
} from '@solana/kit';
import {getTransferSolInstruction} from '@solana-program/system';

// Inside your component
const {signAndSendTransaction} = useSignAndSendTransaction();
const {wallets} = useWallets();

const selectedWallet = wallets[0];

const amount = 1;

const transferInstruction = getTransferSolInstruction({
  amount: BigInt(parseFloat(amount) * 1_000_000_000), // Convert SOL to lamports
  destination: address('RecipientAddressHere'),
  source: createNoopSigner(address(selectedWallet.address))
});

// Configure your RPC connection to point to the correct Solana network
const {getLatestBlockhash} = createSolanaRpc('https://api.mainnet-beta.solana.com
const {value: latestBlockhash} = await getLatestBlockhash().send();

// Create a transaction using @solana/kit
const transaction = pipe(
  createTransactionMessage({version: 0}),
  (tx) => setTransactionMessageFeePayer(address(selectedWallet.address), tx), // S
  (tx) => setTransactionMessageLifetimeUsingBlockhash(latestBlockhash, tx), // Se
  (tx) => appendTransactionMessageInstructions([transferInstruction], tx), // Add
  (tx) => compileTransaction(tx), // Compile the transaction
```

```
  (tx) => new Uint8Array(getTransactionEncoder().encode(tx)) // Finally encode the
```

privy

```
// Send the transaction
const result = await signAndSendTransaction({
  transaction: transaction,
  wallet: selectedWallet
});

console.log('Transaction sent with signature:', result.signature);
```

## Parameters

Hide parameters

**transaction** `Uint8Array` `required`

The encoded transaction to be sent.

**wallet** `ConnectedStandardSolanaWallet` `required`

The Solana wallet to use for sending the transaction.

**chain** `SolanaChain`

Type of all Solana chains supported by Privy.

**options**

`SolanaSignAndSendTransactionOptions & {uiOptions?: SendTransactionModalUIOptions; sponsor?: boolean}`

Additional options for sending the transaction.

Show child attributes

Returns

**signature** `Uint8Array`

The signature of the transaction.

## Sign and send all transactions

To sign and send multiple transactions in a single call, use the `signAndSendTransaction` method with multiple inputs:

```
signAndSendTransaction(...inputs: SignAndSendTransactionInput[]): P
```

### Usage

```
import {useSignAndSendTransaction, useWallets} from '@privy-io/reac
import {
  pipe,
  createSolanaRpc,
  getTransactionEncoder,
  createTransactionMessage,
  setTransactionMessageFeePayer,
  setTransactionMessageLifetimeUsingBlockhash,
  appendTransactionMessageInstructions,
  compileTransaction,
  address,
  createNoopSigner
} from '@solana/kit';
import {getTransferSolInstruction} from '@solana-program/system';

// Inside your component
const {signAndSendTransaction} = useSignAndSendTransaction();
const {wallets} = useWallets();

const selectedWallet = wallets[0];

const amount = 1; // Amount of SOL to send

const transferInstruction = getTransferSolInstruction({
  amount: BigInt(amount * 1_000_000_000), // Convert SOL to lamports
  destination: address('RecipientAddressHere'),
  source: createNoopSigner(address(selectedWallet.address))
});

// Configure your RPC connection to point to the correct Solana network
const {getLatestBlockhash} = createSolanaRpc('https://api.mainnet-beta.solana.com
const {value: latestBlockhash} = await getLatestBlockhash().send();

// Create transactions using @solana/kit
const transactions = [
  pipe(
    createTransactionMessage({version: 0}),
    (tx) => setTransactionMessageFeePayer(address(selectedWallet.address), tx), //
    (tx) => setTransactionMessageLifetimeUsingBlockhash(latestBlockhash, tx), // S
    (tx) => appendTransactionMessageInstructions([transferInstruction], tx), // Ad
```

```
    (tx) => compileTransaction(tx), // Compile the transaction
    (tx) => new Uint8Array(getTransactionEncoder().encode(tx)) // Finally encode
  ),
  pipe(
    createTransactionMessage({version: 0}),
    (tx) => setTransactionMessageFeePayer(address(selectedWallet.address), tx),
    (tx) => setTransactionMessageLifetimeUsingBlockhash(latestBlockhash, tx),
    (tx) => appendTransactionMessageInstructions([transferInstruction], tx),
    (tx) => compileTransaction(tx),
    (tx) => new Uint8Array(getTransactionEncoder().encode(tx))
  )
];

// Send multiple transactions
const results = await signAndSendTransaction(
  {
    transaction: transactions[0],
    wallet: selectedWallet
  },
  {
    transaction: transactions[1],
    wallet: selectedWallet
  }
);

console.log(
  'Transactions sent with signatures:',
  results.map((result) => result.signature.toString()).join(',')
);
```

## Parameters

Hide parameters

**inputs** `SignAndSendTransactionInput[]` `required`

**transaction** `Uint8Array` `required`

The encoded transaction to be sent.



**wallet** `ConnectedStandardSolanaWallet` `required`

The Solana wallet to use for sending the transaction.

**chain** `SolanaChain`

Type of all Solana chains supported by Privy.

**options**

`SolanaSignAndSendTransactionOptions & {uiOptions?: SendTransactionModalUIOptions; sponsor?: boolean}`

Additional options for sending the transaction.

Show child attributes

For complete examples of sending SOL and SPL tokens using Privy's SDKs, check out the **sending a SOL transaction recipe** and the **sending SPL tokens recipe**.

Was this page helpful? Yes No



Powered by mintlify